Technical Case Study

AI Voice Agent

Deadline	17.03.2026 — 18:00 CET (if you require an extension, please notify us in advance)
Delivery	GitHub repo + technical documentation (README or separate .md)
AI Usage	Permitted — maximum 40–50%
Language	English required — German is a plus
Required	TypeScript (no other language accepted)
Required	Apple Silicon (M-series Mac) — the entire pipeline must run on Apple Silicon

Scenario

An insurance company wants to automate its customer service operations. Their call center currently handles around 300 inbound calls per day. 60% of these calls are concentrated in the following three topics:

- Policy status enquiry
- Initiating a damage/claim report
- Scheduling or cancelling an appointment

The company wants an AI voice agent capable of handling these calls. The agent must support English; German support is a plus. It should understand the customer's intent, determine the correct action, and — where applicable — simulate the next step (e.g. creating a claim record).

You are tasked with building the core voice + LLM pipeline for this system.

Tasks

Task 1 — Voice Pipeline

Accept audio input from the user, transcribe it to text, process it with an LLM, and convert the response back to speech. A real telephony infrastructure is not required; microphone or audio file input is sufficient.

- STT: Whisper (local) or an alternative of your choice
- TTS: ElevenLabs, OpenAI TTS, or macOS native speech API
- The entire pipeline must be written in TypeScript
- Latency must be minimised — elapsed time (ms) between each step must be reflected in the logs

Task 2 — LLM Integration & Intent Detection

Build an intent system that analyses incoming speech and triggers the correct action.

- Define at least 3 intents: policy_enquiry, report_claim, schedule_appointment
- Implement a graceful fallback for unrecognised intents
- LLM choice is open (OpenAI, Anthropic, local model — justify your decision in the documentation)
- Document your prompt design and versioning approach

Task 3 — Memory & Conversation Context

The agent must remember what was said within the same session.

- Manage conversation history across turns
- Once the customer states their name, the agent must recall it in subsequent turns
- Demonstrate that context is cleared when the session ends
- Optional bonus: basic cross-session memory mechanism (JSON file or SQLite)

Task 4 — Technical Documentation

The following must be included in a technical document submitted alongside the project:

- System architecture — what each component does (diagram or written description)
- Setup and run instructions (README)
- Justification for your LLM and STT/TTS choices
- Prompt design and version notes
- Known limitations and suggestions for improvement
- AI usage declaration — which parts used AI, and to what extent?

Technical Constraints

> ⚠ Apple Silicon (M-series Mac) is required. The pipeline must run on Apple Silicon and the documentation must include output/logs confirming this.
> If you do not have access to an Apple Silicon device, you must notify us before starting the case study so we can discuss alternatives.

⚠ TypeScript is required. Submissions in JavaScript, Python, or any other language will not be considered.

⚠ AI usage is capped at 40–50%. At least 50% of all code and documentation must be the candidate's own work. Submissions without an AI usage declaration are not accepted.

⚠ A GitHub repository is required. Commit history is part of the evaluation. Repositories with a single "initial commit" will not be accepted.

Delivery Requirements

- Public or private GitHub repository (if private, send an access invitation)
- Commit history must be readable and meaningful (not single-word commits like "fix" or "wip")
- README: setup, dependencies, and run command
- TECHNICAL_DOC.md or a docs/ folder containing the technical documentation
- AI_USAGE.md: declaration of where and to what extent AI was used

Evaluation Criteria

Criterion	Weight	Description
Working Pipeline	**25%**	Does the voice → STT → LLM → TTS flow work end-to-end?
Code Quality	**20%**	TypeScript types, modular structure, readability
LLM & Intent Design	**20%**	Prompt engineering, edge case handling, fallback
Memory Management	**15%**	Is context maintained correctly? Is session management solid?
Technical Documentation	**15%**	Is the architecture clear, setup working, decisions justified?
AI Usage Declaration	**5%**	Is it honest, clear, and does it specify the percentage?

Bonus (Optional)

Adding one or more of the following will be noted positively in the evaluation:

- Cross-session persistent customer memory
- A simple CLI or web interface
- Latency benchmarking report
- German language support

Good luck. 🚀